```
class UploadController < ApplicationController
  def get
    @picture = Picture.new
  end
  # . . .
  private
    # Never trust parameters from the scary internet, only allow the white
    # list through.
    def picture_params
      params.require(:picture).permit(:comment, :uploaded_picture)
    end
end
```

The `get` template contains the form that uploads the picture (along with a comment). Note how we override the encoding type to allow data to be sent back with the response:

rails7/e1/views/app/views/upload/get.html.erb

```
<%= form_for(:picture,
             url: {action: 'save'},
             html: {multipart: true}) do |form| %>

    Comment:             <%= form.text_field("comment") %><br/>
    Upload your picture: <%= form.file_field("uploaded_picture") %><br/>

    <%= submit_tag("Upload file") %>
<% end %>
```

The form has one other subtlety. The picture uploads into an attribute called `uploaded_picture`. However, the database table doesn't contain a column of that name. That means that there must be some magic happening in the model:

rails7/e1/views/app/models/picture.rb

```
class Picture < ActiveRecord::Base

  validates_format_of :content_type,
                      with: /\Aimage/,
                      message: "must be a picture"

  def uploaded_picture=(picture_field)
    self.name         = base_part_of(picture_field.original_filename)
```